SkillSwap - informe de decisiones de diseño

Trabajo Practico Final - Proceso de Desarrollo de Software

Decisiones arquitectonicas, reglas de dominio y evidencia

Trabajo Práctico Final - Proceso de Desarrollo de Software Tecnicatura en Desarrollo de Software Corte revisado: **main** en **36b07b0**, 26/10/2026. Los cambios de este informe y del modelo se proponen en un PR separado.

1. Objetivo y alcance

SkillSwap es un marketplace académico de mentorías. Este informe explica la ubicación de responsabilidades, las reglas que se protegen en el dominio y los patrones que ya tienen un caso concreto. El proyecto ejecutable que existe hoy es un Mock UI Kit de escritorio. Los contratos de Application y el modelo de dominio están integrados, pero los services de casos de uso, los repositorios JPA y la integración real de Presentation todavía no.

El informe no presenta el diseño futuro como funcionalidad terminada. Las decisiones se contrastan con el código disponible en el corte indicado.

2. Arquitectura por capas

Capa	Responsabilidad	Evidencia y límite actual
Presentation	Recibir acciones del usuario, convertir formatos de interfaz y delegar.	**ui/SkillSwapView**, **controllers/UiControllers** y **mock/MockSkillSwapControllers**; el controlador actual es un mock.
Application	Exponer puertos de entrada/salida y coordinar un caso de uso, seguridad del actor, transacciones y colaboraciones.	Hay contratos y modelos bajo **application/**. Los services concretos todavía no están implementados.
Domain	Proteger invariantes del negocio mediante agregados, value objects, estados y políticas.	**domain/model**, **domain/state**, **domain/policy** y **domain/pricing**; no depende de framework o almacenamiento.
Infrastructure	Implementar persistencia, transacciones, migraciones y clientes externos requeridos por los puertos.	Hibernate, MySQL y Flyway aparecen como dependencias previstas; los adapters concretos siguen pendientes.

La dirección de dependencia entra hacia el dominio: Presentation consume puertos de entrada de Application; un service implementa esos casos y depende de puertos de salida; Infrastructure implementa esos puertos. La configuración externa ensambla las implementaciones. Domain no conoce controllers, services, repositorios concretos, DTOs, JPA/Hibernate ni APIs externas.

Responsabilidad de los futuros services

Los contratos actuales son **ProfileUseCases**, **PublicationUseCases**, **DiscoveryUseCases**, **BookingUseCases**, **WalletUseCases**, **RatingUseCases** y **NotificationUseCases**. Las implementaciones deben seguir esa separación de capacidades.

Service	Orquestación a su cargo	Regla que permanece en Domain
ProfileService	Identificar al actor, cargar/guardar perfil y proyectar **ProfileView**.	Titularidad, perfil activo, campos, disponibilidad y reputación derivada.
PublicationService	Crear/cargar/guardar publicaciones y convertir comandos/resultados.	Edición de borrador, permisos, disponibilidad y transiciones de estado.
DiscoveryService	Validar contexto, consultar resultados paginados y aplicar recomendación elegida.	Criterios de recomendación delegados a **RecommendationStrategy**.
BookingService	En una transacción, verificar actor, reservar franja, cambiar wallet y persistir el booking.	Participantes, transiciones, pago permitido, cancelación, reintegro y reprogramación.
WalletService	Leer la billetera del usuario autenticado y mapear el resultado.	Reglas de saldo, monto y registro de movimientos.
RatingService	Verificar unicidad con el repositorio, cargar booking/perfil y actualizar datos relacionados atómica y consistentemente.	Contratación completada, autor participante, puntaje y promedio del perfil.
NotificationService	Consultar/actualizar bandeja y despachar notificación/eventos mediante puertos.	Preferencias y tipos de canal permanecen en el modelo de negocio.

Un service no repite validaciones que ya protegen las entidades ni concentra reglas de todos los agregados. Sus transacciones y repositorios son puertos; JPA, locks y APIs externas no se filtran al dominio.

3. Expertos de información y reglas de negocio

Tipo del dominio	Regla que conoce/protege	Colaboración principal
Profile	Solo su titular activo puede modificar el perfil; normaliza habilidades/intereses; administra preferencias y disponibilidad; calcula reputación a partir de calificaciones recibidas.	**AvailabilitySlot**, **Rating**.
AvailabilitySlot	Valida intervalo y capacidad; calcula lugares restantes.	**Publication** y reserva de **Booking**.
Publication	Protege el borrador, publicación/pausa/cierre, permisos del autor, disponibilidad y solicitud de reserva.	**PublicationState**, **AvailabilitySlot**, **SkillCoins**.
PublicationBuilder	Acumula los datos necesarios y produce un borrador válido.	Crea **Publication** y disponibilidad asociada.
Booking	Verifica participantes, transiciones permitidas, pago, cancelación, reintegro y acuerdo de ambas partes para reprogramar; conserva historial.	**BookingState**, **Publication**, **StatusHistoryEntry**, **SkillCoins**.
Wallet y **SkillCoins**	Evitan montos/saldos inválidos y representan créditos, débitos y movimientos.	**LedgerEntry**; Application coordina cambios de dos billeteras en una transacción.
Rating	Solo permite calificar una contratación completada y por uno de sus participantes; valida el puntaje.	**Booking**, **Profile**. La unicidad persistente se coordina fuera de la entidad.

Tipo del dominio	Regla que conoce/protege	Colaboración principal
PriceQuote y políticas	Calculan una cotización con promociones; **RefundPolicy** calcula reintegros según su regla.	**PriceDecorator**, promociones y estrategias de reintegro.

Esta asignación sigue Information Expert: la entidad que tiene la información valida sus propios invariantes. El caso de uso puede combinar entidades y fuentes, pero no sustituir ese conocimiento.

4. Patrones con necesidad concreta

Patrón	Necesidad y colaboración	Evidencia en main
Strategy	Cambia el algoritmo de recomendación o política de reintegro sin cambiar su consumidor.	**RecommendationStrategy** con afinidad/popularidad; **RefundPolicy** con reglas estándar/flexible; **DomainRulesTest**.
State	El conjunto de acciones permitidas depende del estado de una publicación o contratación.	**PublicationState** / **BookingState**, implementaciones en **PublicationStates** / **BookingStates** y pruebas de transiciones.
Builder	Una publicación tiene datos obligatorios y opcionales y se debe construir solo cuando está completa.	**PublicationBuilder** y **PublicationBuilderTest**.
Decorator	Una cotización recibe promociones acumulables sin alterar el cálculo del precio base.	**PriceQuote**, **PriceDecorator**, **FixedAmountPromotion**, **PercentagePromotion**, **PromotionDecoratorTest**.

Estos cuatro patrones GoF están implementados y probados en el alcance actual, cumpliendo el mínimo de la consigna. El **Singleton** existe de forma localizada en enums de estado: cada estado es una instancia inmutable. No se agrega un Singleton mutable de aplicación.

Patrones registrados sin implementación artificial

- Adapter: todavía no hay SDK/proveedor externo incompatible que adaptar. **NotificationGateway** y **CurrentUserProvider** son puertos requeridos por Application; cuando se incorpore un proveedor concreto, un adapter en Infrastructure traducirá su interfaz. Un puerto no es, por sí solo, un Adapter.
- Observer: **DomainEventPublisher** solo define un puerto. El patrón requiere suscriptores y notificación desacoplada a varios interesados; eso aún no está implementado.
- Simple Factory / Factory Method / Abstract Factory: no hay aún una selección real de canales ni una familia de proveedores que justifique fábricas. Reconsiderar cuando haya variantes concretas, no crear capas de construcción vacías.
- Composite: no hay una jerarquía de categorías/habilidades compuestas que se recorra uniformemente.
- Facade: no hay una consulta integrada sobre varias fuentes que necesite simplificarse.
- MVC: el Mock UI Kit separa vista, contrato de controller y mock. Falta su controller real hacia los casos de uso.
- Repository / DAO: están los contratos de repositorio en Application; JPA/DAO concretos y sus pruebas están pendientes. DAO se registra como técnica de acceso a datos, no como GoF.
- CQRS: los puertos separan capacidades y nombran comandos/consultas, pero aún no existe una separacion formal de modelos de lectura/escritura o de almacenamientos.

5. GRASP, SOLID y decisiones de colaboración

- Creator: **Profile.register**, **Publication.draft**, **Booking.request** y **Wallet.open** construyen objetos desde la información que requieren; **PublicationBuilder** maneja su construcción gradual.
- Controller: el controller de Presentation recibe la acción del usuario y delega; el mock actual solo permite demostrar el flujo simulado.

- Polymorphism: State, Strategy y Decorator varian comportamiento mediante contratos.
- Low Coupling / Protected Variations: la UI conoce un contrato; Application publica puertos; el dominio depende de Java estándar. Cambiar MySQL o un proveedor no debería cambiar las entidades.
- High Cohesion: cada agregado protege reglas de un área; cada puerto agrupa capacidades relacionadas; mappers y adaptadores quedan fuera de las entidades.
- Single Responsibility: entidades, contratos, services, controllers y adapters tienen responsabilidades diferentes. Un service coordina, no hace persistencia directamente ni contiene todas las reglas.
- Open/Closed y Liskov: nuevas estrategias, estados o promociones deben cumplir su contrato y poder agregarse sin invalidar consumidores existentes; las pruebas de dominio cubren las variantes actuales.
- Interface Segregation: los puertos se separan por área funcional en lugar de un **SkillSwapService** con todas las operaciones.
- Dependency Inversion: Application declara interfaces que necesita; Infrastructure debe implementarlas y depender hacia adentro. El wiring concreto sigue pendiente.

Los DTOs/modelos de Application (**ProfileData**, **BookingData**, etc.) son distintos de las entidades; la presentación debe recibir modelos de respuesta y nunca la entidad de dominio directamente. Los controllers convierten y delegan; las implementaciones JPA traducen entre persistencia y agregados. No se ubican DTOs dentro de entidades ni dependencias de framework en Domain.

6. Riesgos y trabajo pendiente

1. Implementar services de Application y probar orquestación mediante puertos simulados. 2. Implementar repositorios y mapeos JPA, esquema y migraciones MySQL/Flyway. 3. Definir transacciones y lectura/bloqueo para evitar dos reservas de la última franja, doble pago y calificaciones duplicadas. 4. Conectar controllers reales al Mock UI Kit, manteniendo las conversiones fuera del dominio. 5. Agregar pruebas de persistencia e integración y generar un reporte de cobertura reproducible. 6. Revisar Adapter, Observer y fábricas solo cuando se elija un proveedor o flujo que demuestre su necesidad.

7. Guía breve para la defensa oral

¿Por qué no se valida todo en el service? Porque las invariantes pertenecen a las entidades que tienen esos datos. El service coordina el caso de uso y delega el cambio al agregado.

¿Qué está completo y qué no? El dominio y los puertos/modelos de Application; el ejecutable sigue siendo una demo en memoria. No se presenta JPA ni los services concretos como implementados.

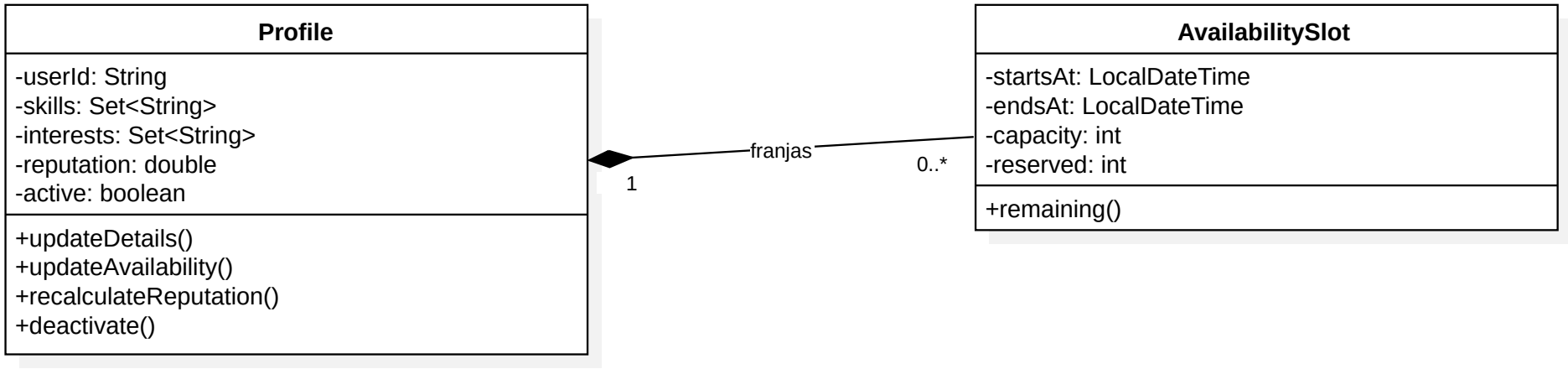
¿Por qué usar Strategy? Recomendación y reintegro tienen reglas alternativas reales, seleccionables por el consumidor sin modificar la entidad.

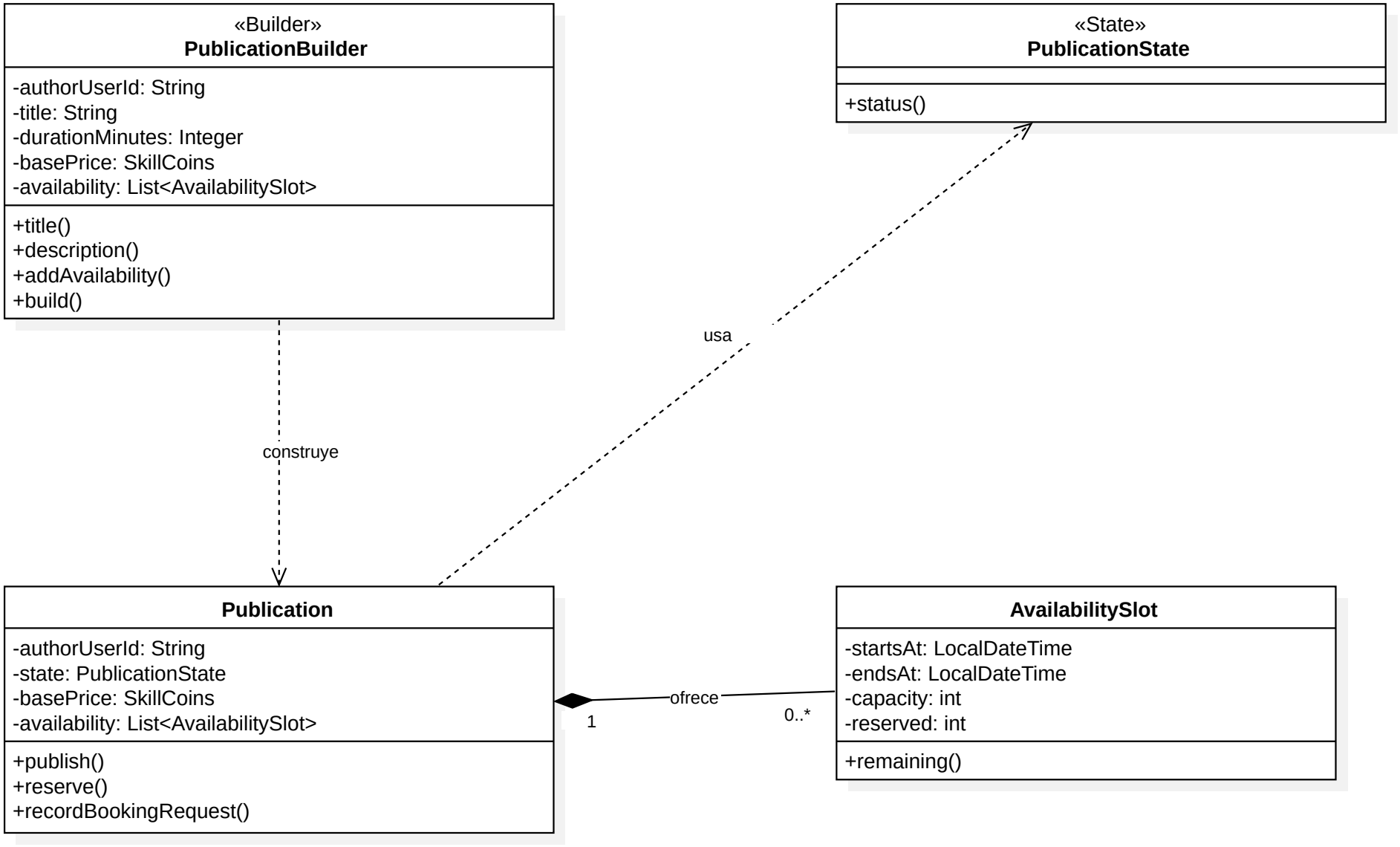
¿Por qué State? Cada estado permite un conjunto distinto de transiciones; cada clase de estado responde si la acción es válida y **Booking**/**Publication** mantiene su estado e historial.

¿Por qué no se implementó Adapter/Observer? No hay proveedor externo con interfaz incompatible ni despacho a suscriptores reales. Los puertos preparan el límite, pero no justifican afirmar que el patrón ya está aplicado.

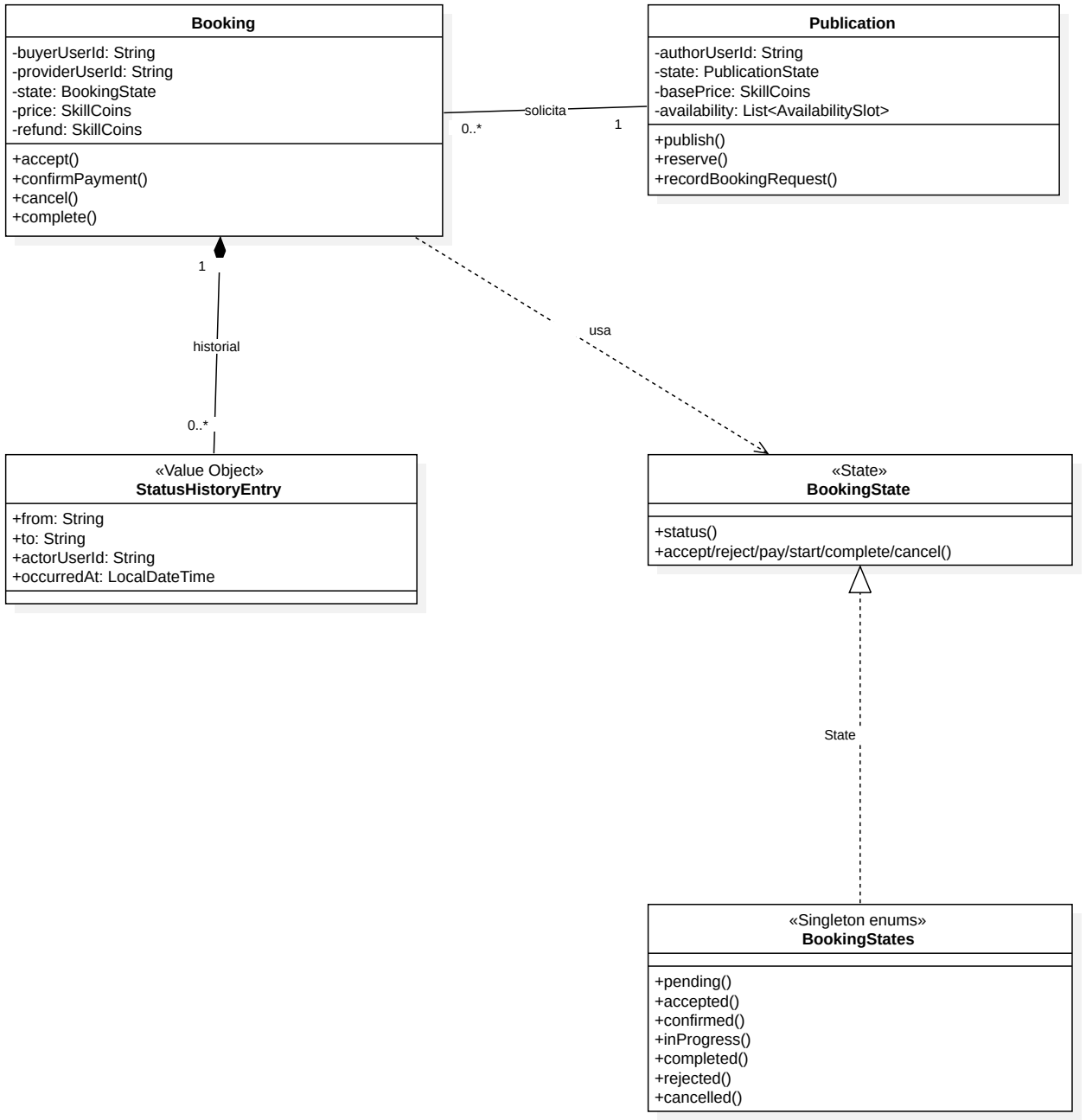
¿Cómo se evita devolver entidades a la UI? Los puertos de entrada usan modelos de Application como **BookingView** y **PublicationView**; el controller convierte solicitudes y respuestas. El PDF adjunta 22 diagramas UML editables desde **docs/modelo-skill-swap.mdj**; muestran contratos existentes y no inventan implementaciones pendientes.

SkillSwap::Dominio - perfil y disponibilidad

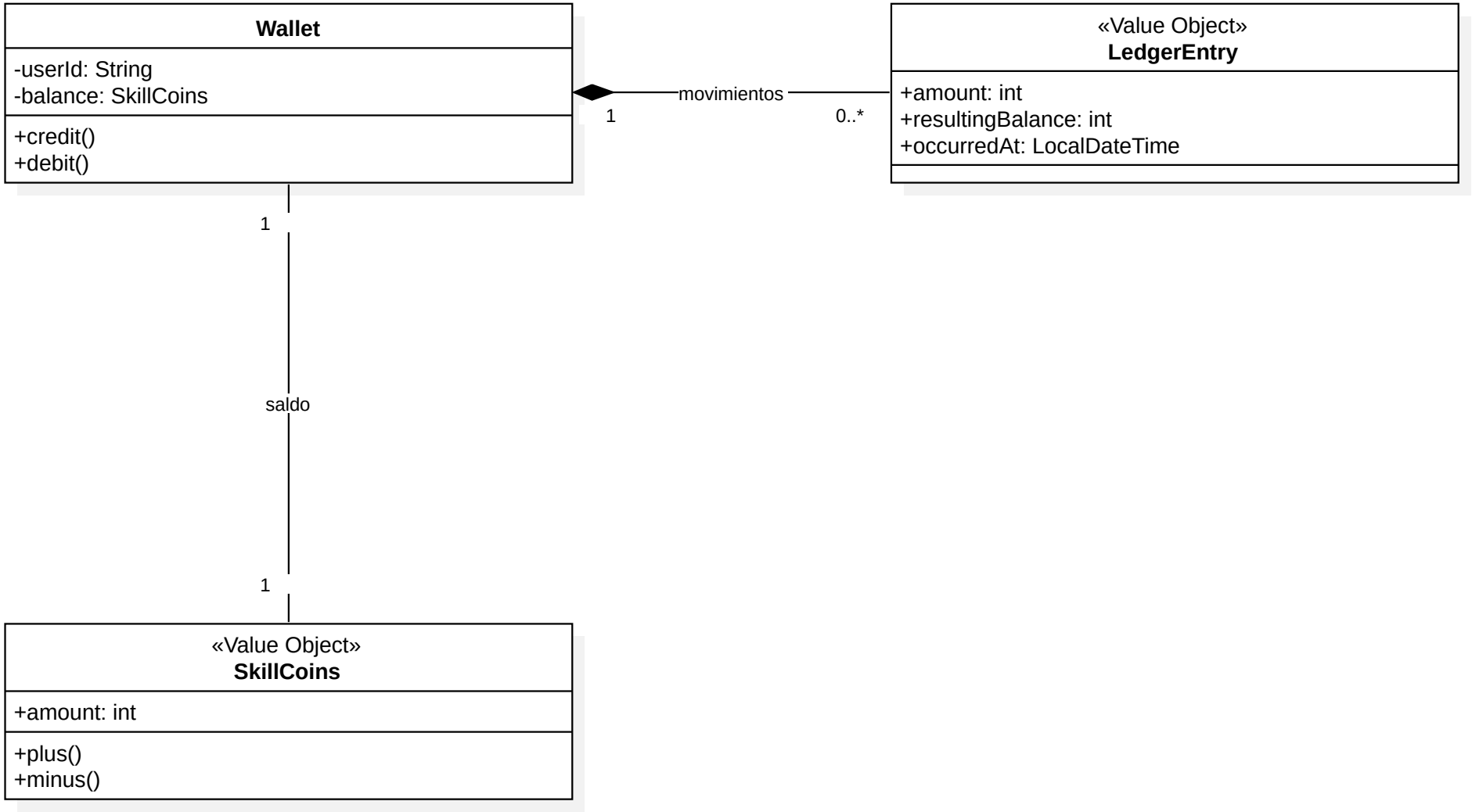



«Builder»
PublicationBuilder
-authorUserId: String
-title: String
-durationMinutes: Integer
-basePrice: SkillCoins
-availability: List<AvailabilitySlot>
+title()
+description()
+addAvailability()
+build()
«State»
PublicationState
+status()
usa
construye
Publication
-authorUserId: String
-state: PublicationState
-basePrice: SkillCoins
-availability: List<AvailabilitySlot>
+publish()
+reserve()
+recordBookingRequest()
ofrece
1
0..*
AvailabilitySlot
-startsAt: LocalDateTime
-endsAt: LocalDateTime
-capacity: int
-reserved: int
+remaining()

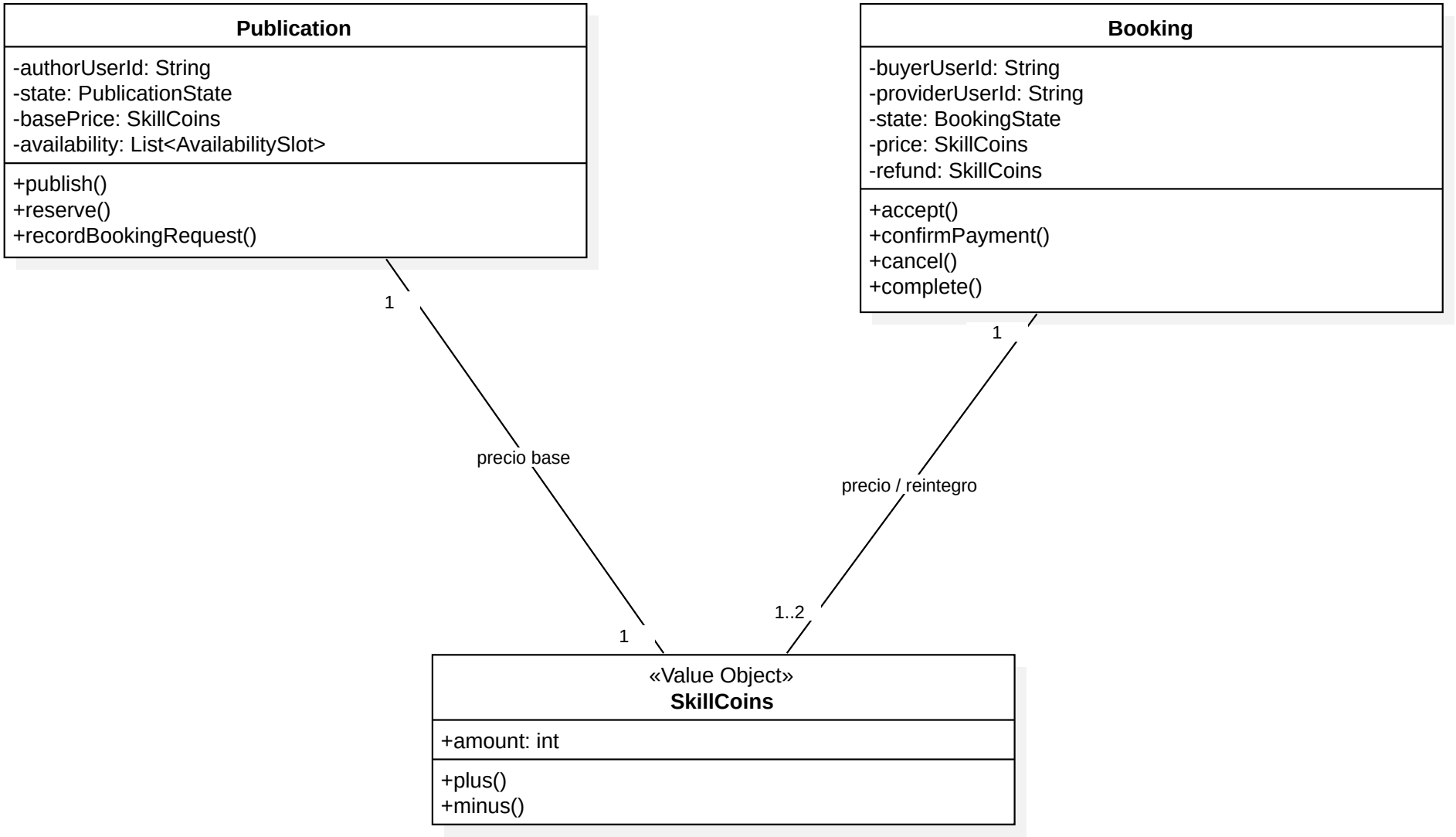
SkillSwap::Dominio - contratación, estado e historial



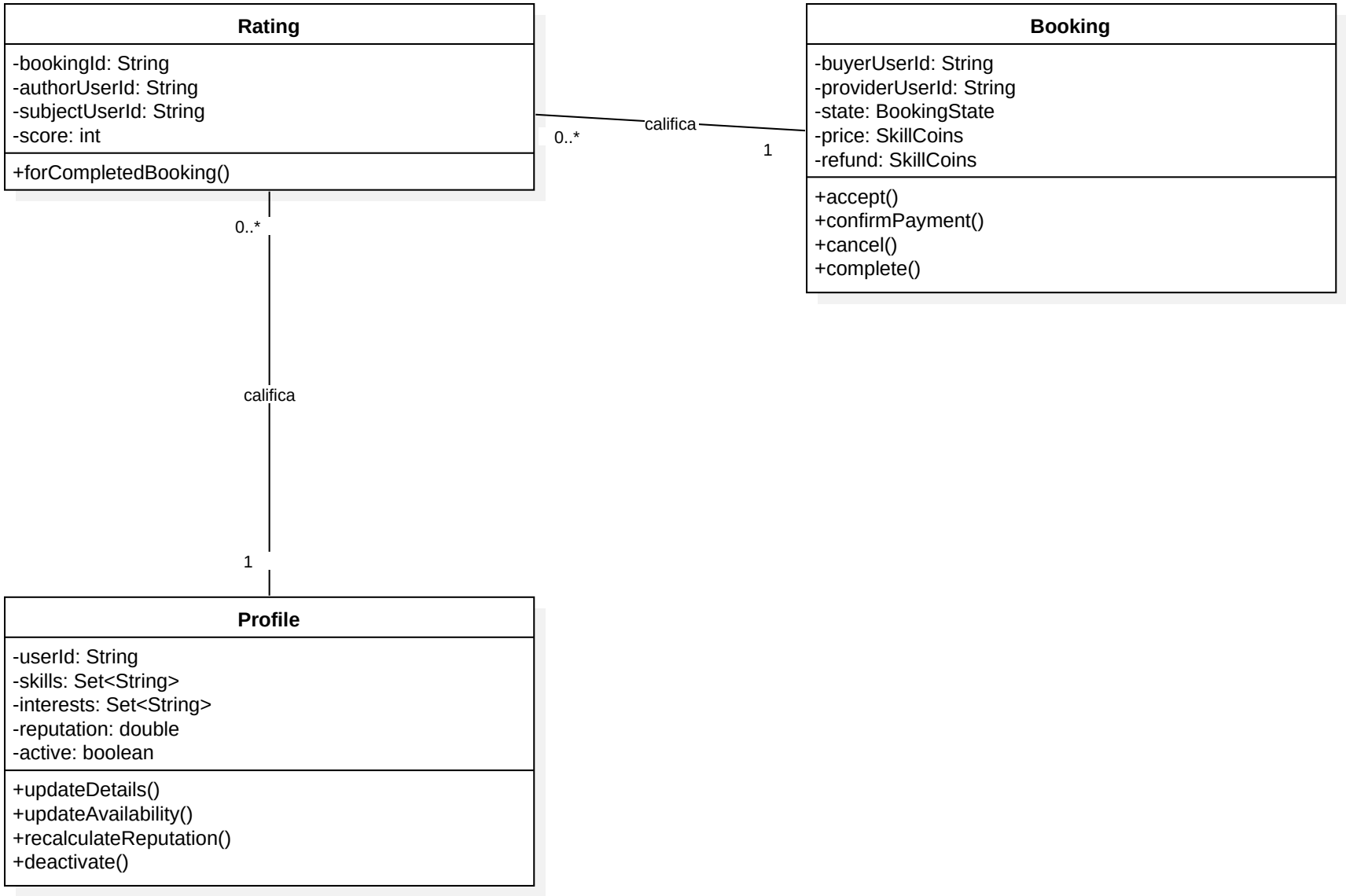
SkillSwap::Dominio - wallet y valor monetario

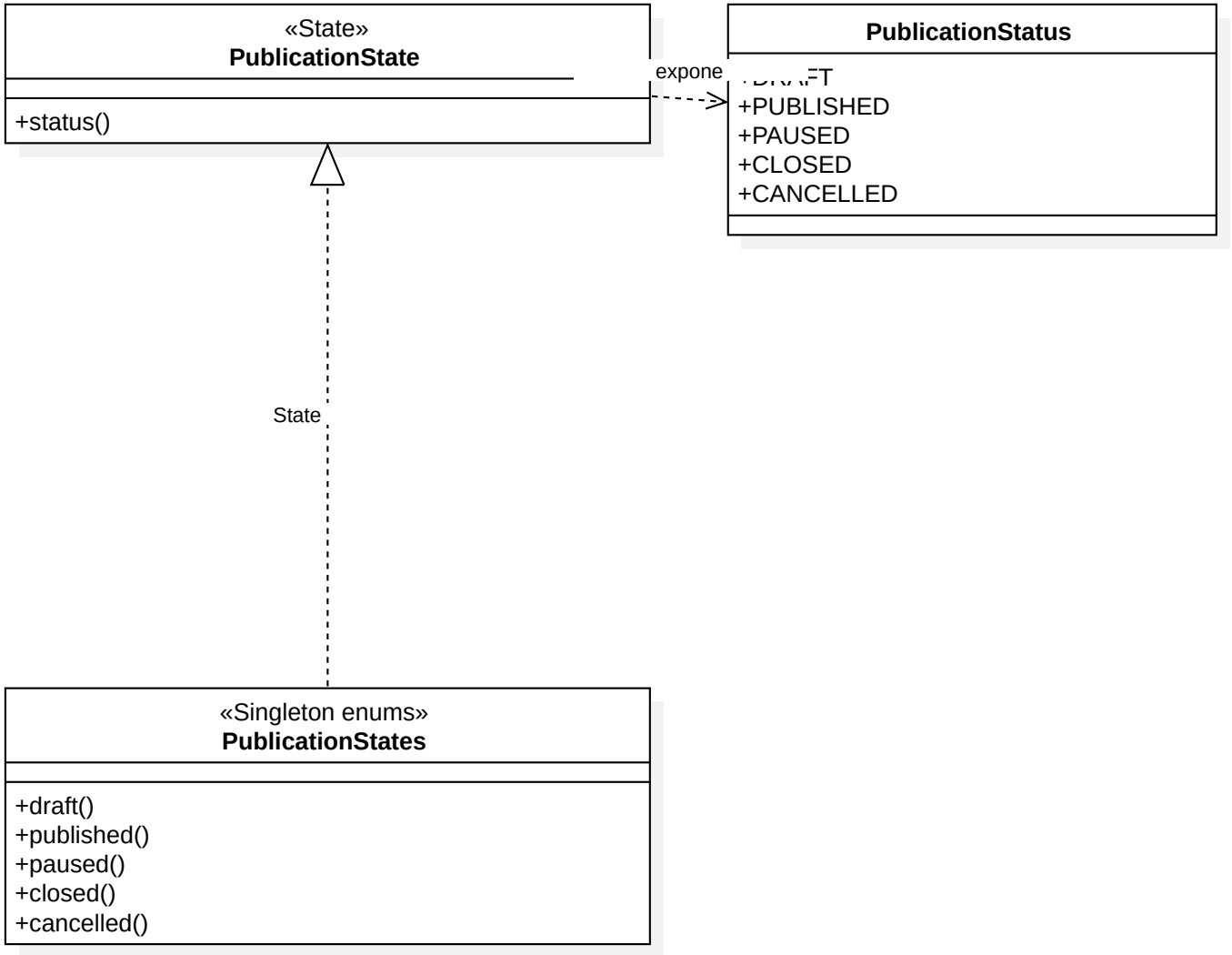


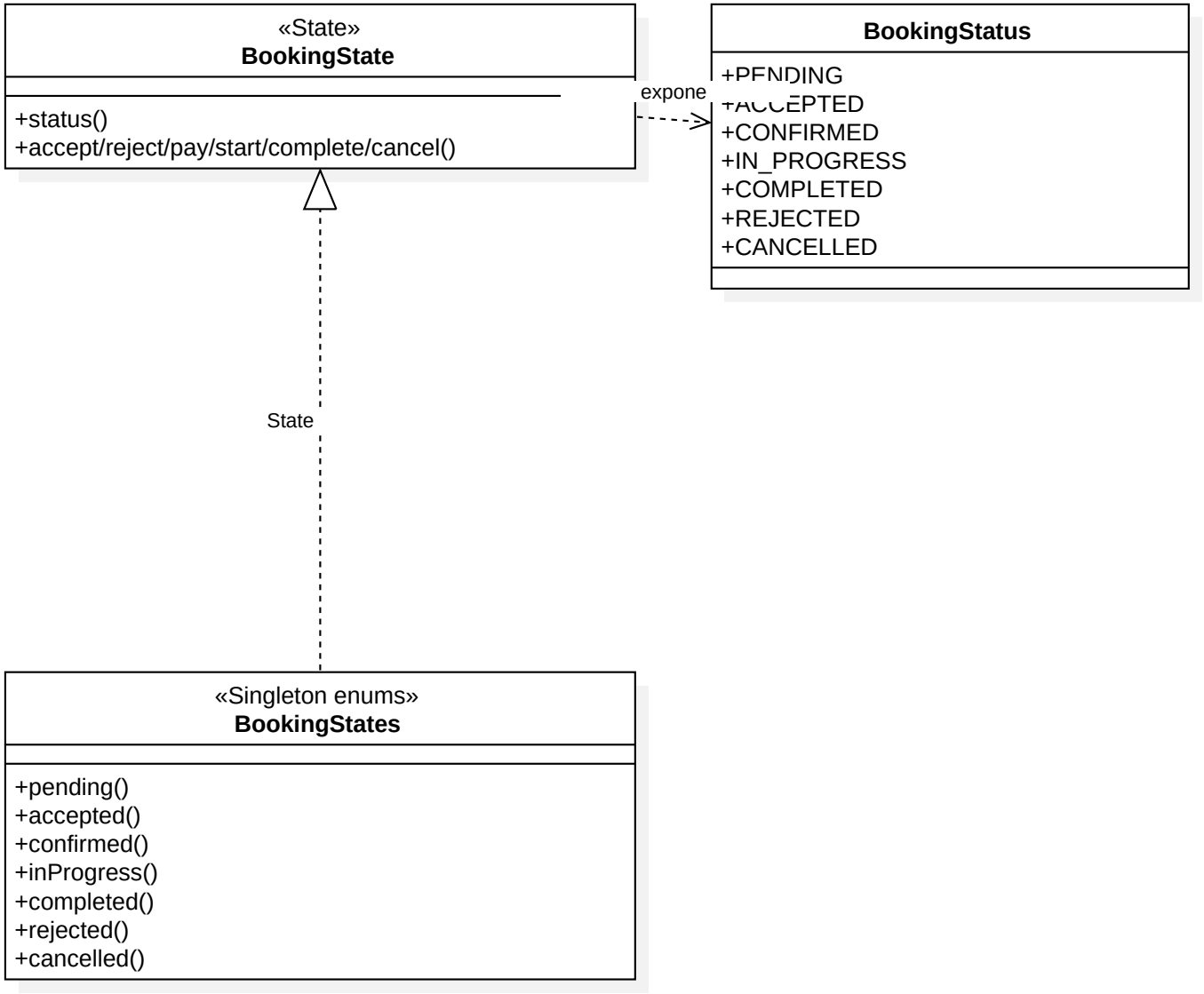
SkillSwap::Dominio - precios de publicaciones y bookings

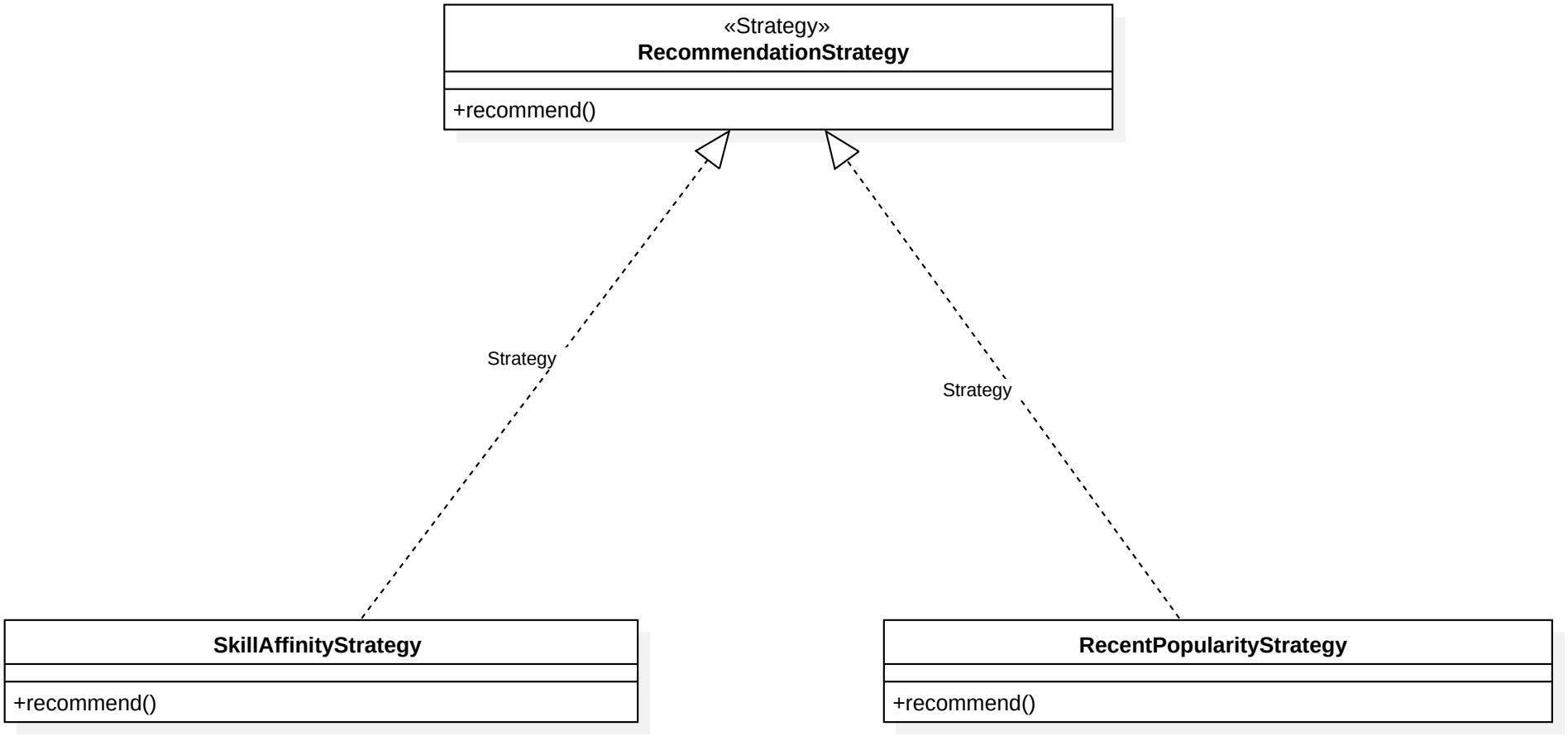


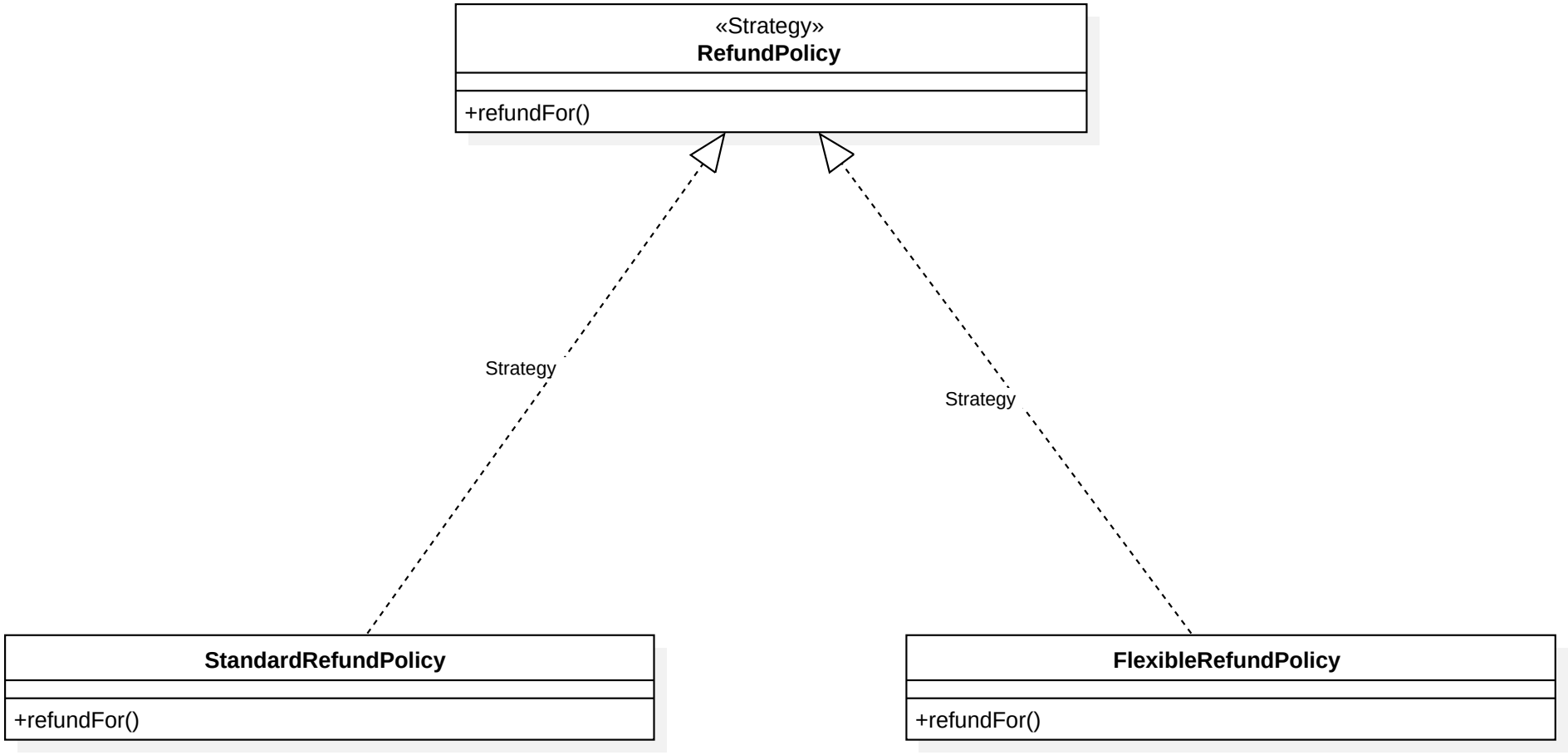
SkillSwap::Dominio - calificaciones y reputación

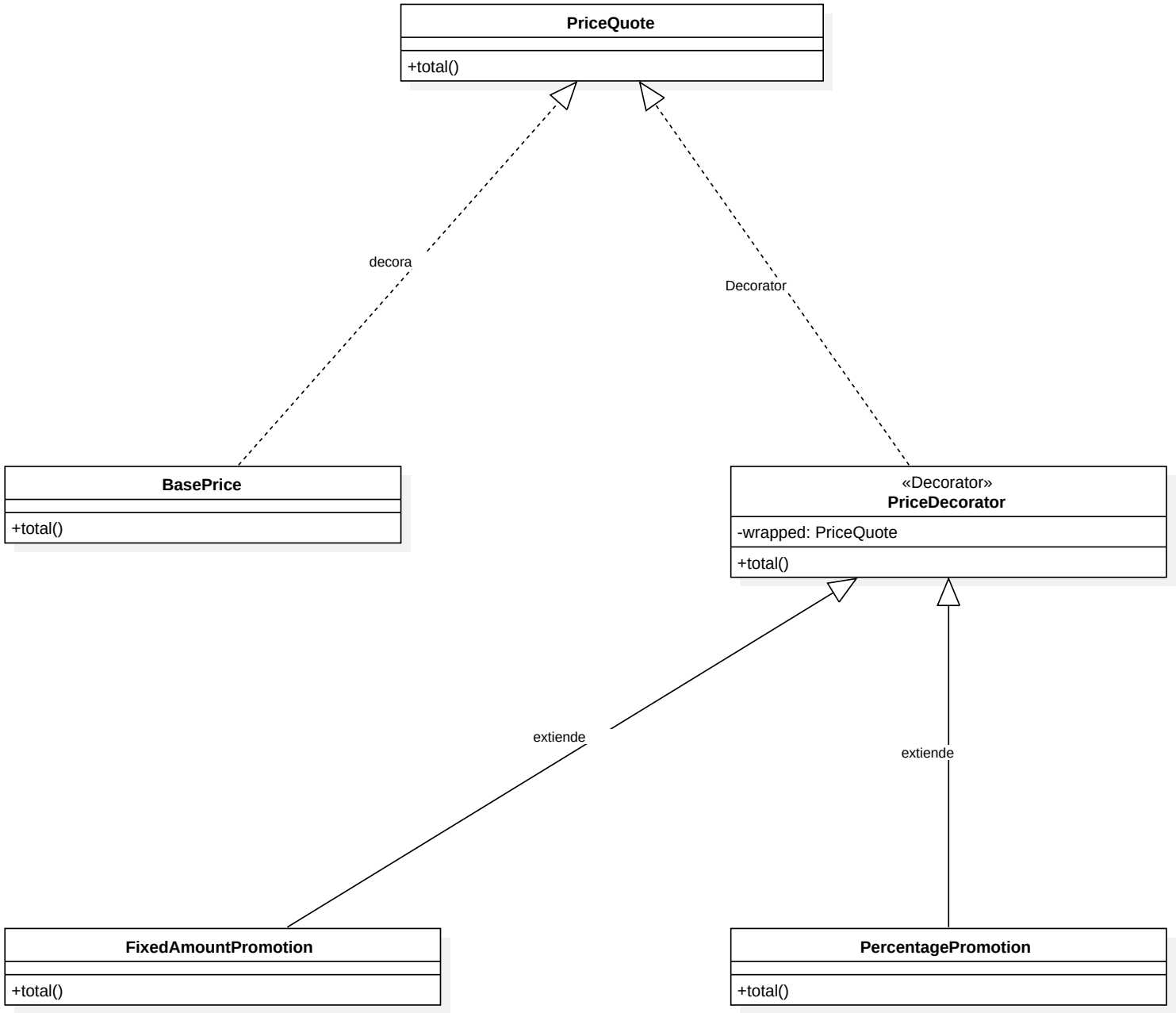



«State»
PublicationState
+status()
expone
PublicationStatus
+DRAFT
+PUBLISHED
+PAUSED
+CLOSED
+CANCELLED
State
«Singleton enums»
PublicationStates
+draft()
+published()
+paused()
+closed()
+cancelled()

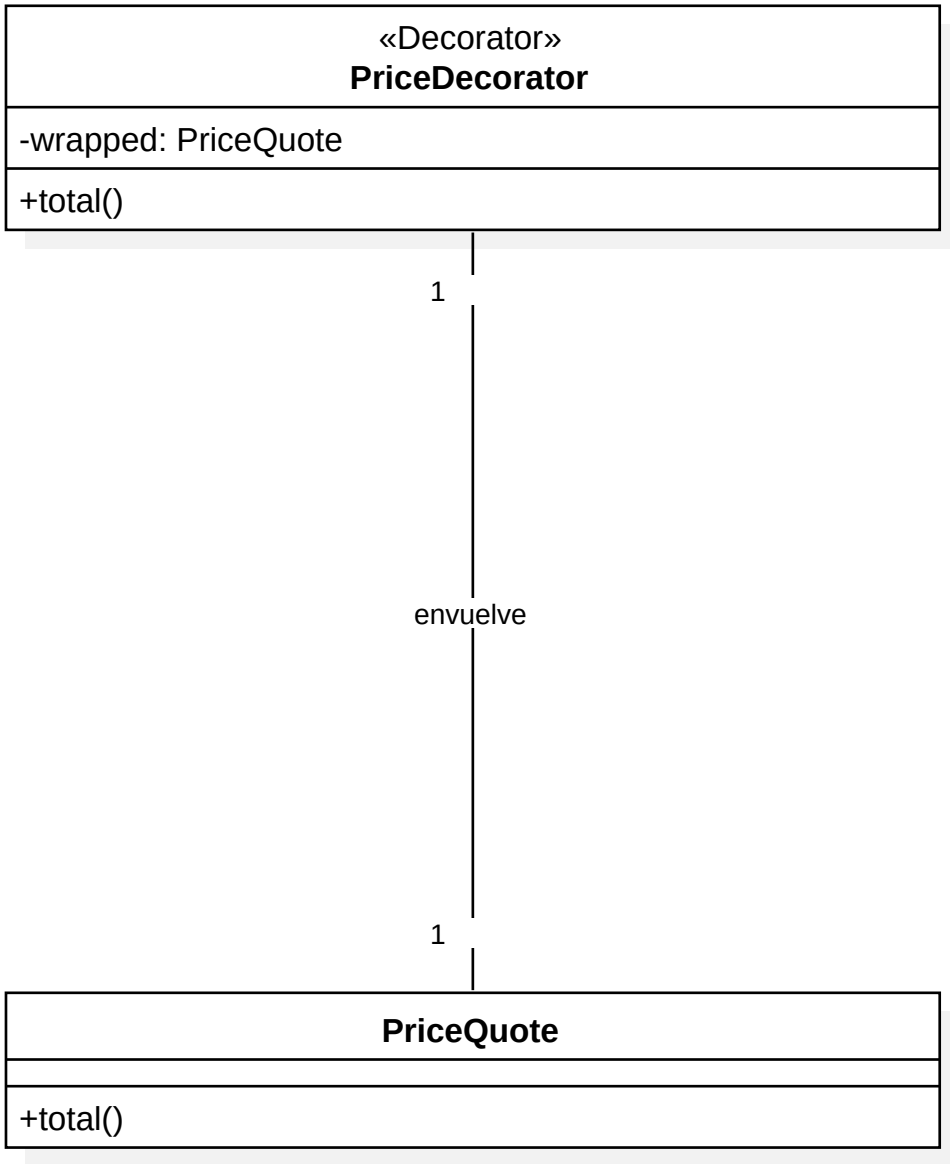

«State»
BookingState
+status()
+accept/reject/pay/start/complete/cancel()
expone
BookingStatus
+PENDING
+ACCEPTED
+CONFIRMED
+IN_PROGRESS
+COMPLETED
+REJECTED
+CANCELLED
State
«Singleton enums»
BookingStates
+pending()
+accepted()
+confirmed()
+inProgress()
+completed()
+rejected()
+cancelled()


«Strategy»
RecommendationStrategy
+recommend()
Strategy
Strategy
SkillAffinityStrategy
+recommend()
RecentPopularityStrategy
+recommend()


«Strategy»
RefundPolicy
+refundFor()
Strategy
Strategy
StandardRefundPolicy
+refundFor()
FlexibleRefundPolicy
+refundFor()


PriceQuote
+total()
decora
Decorator
BasePrice
+total()
«Decorator»
PriceDecorator
-wrapped: PriceQuote
+total()
extiende
extiende
FixedAmountPromotion
+total()
PercentagePromotion
+total()

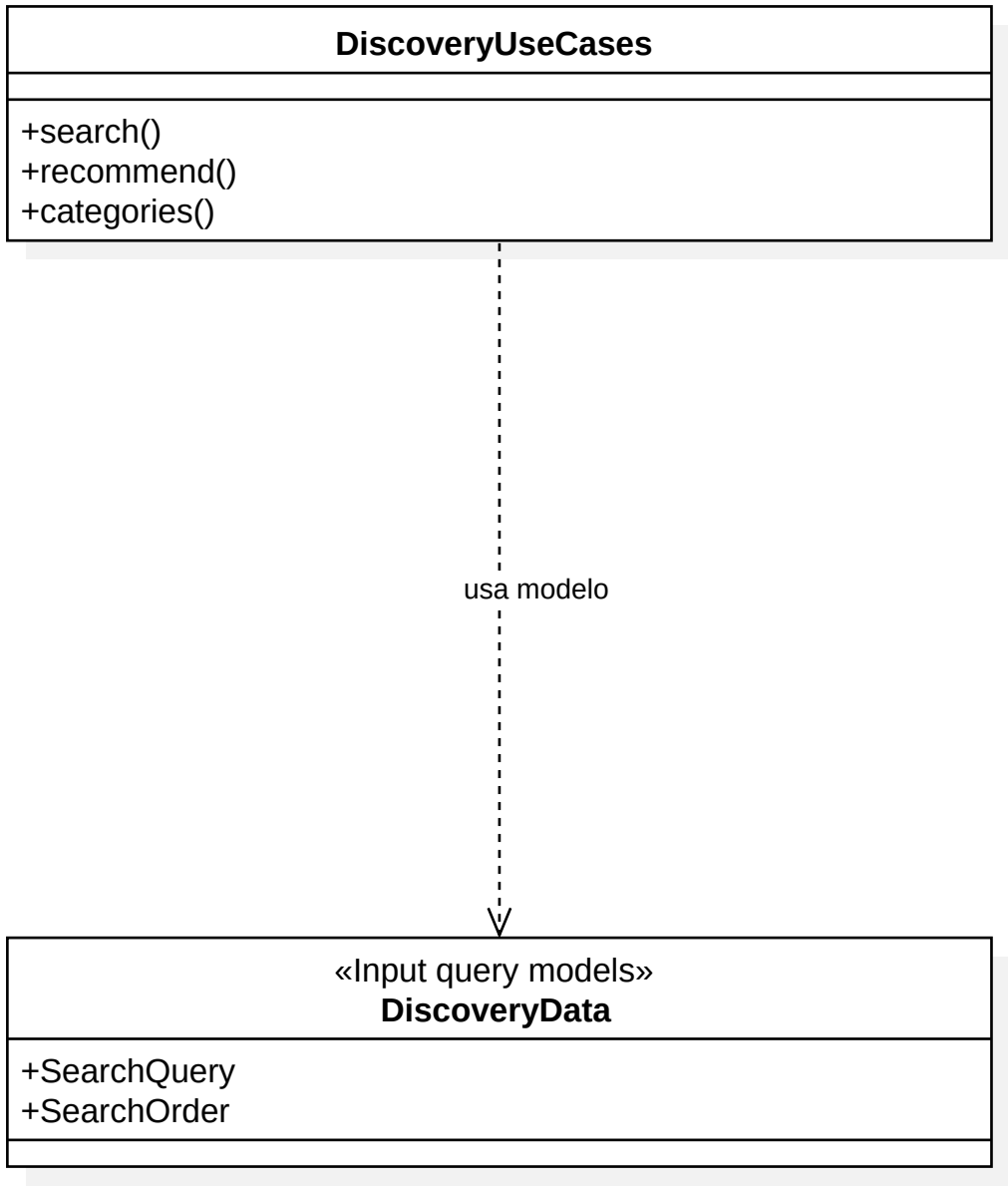
SkillSwap::Patrón Decorator - relación de envoltura

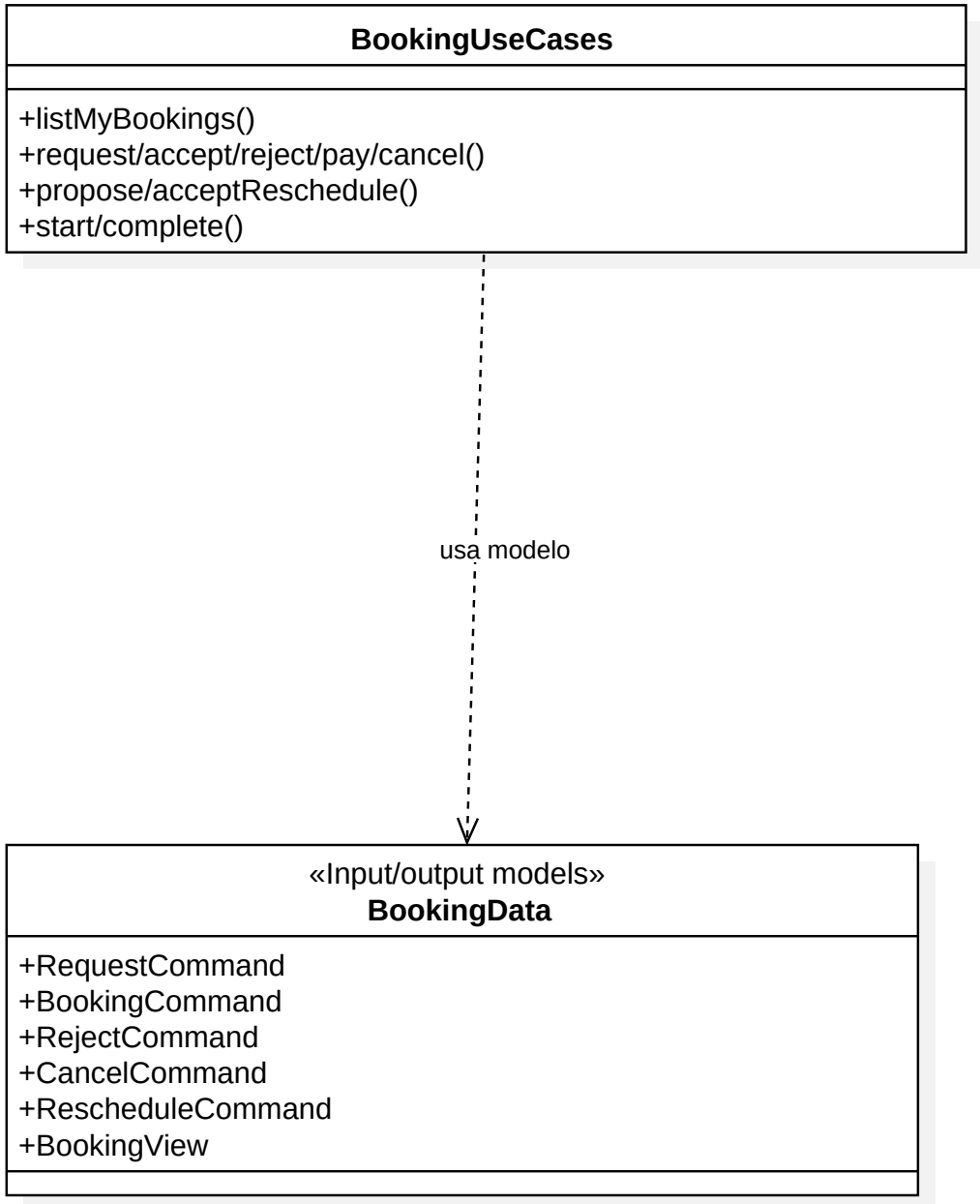


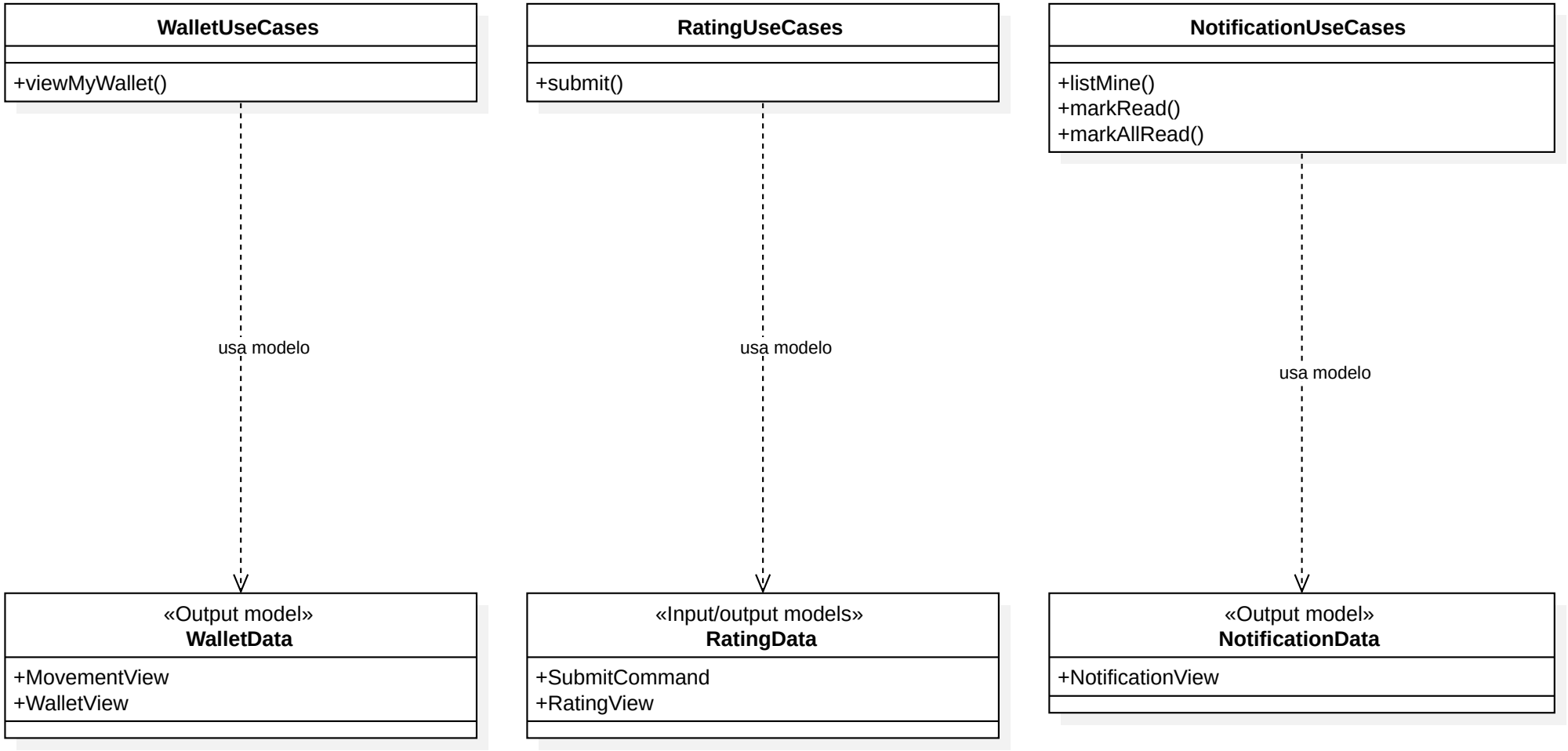
SkillSwap::Application - puertos de perfil y publicación



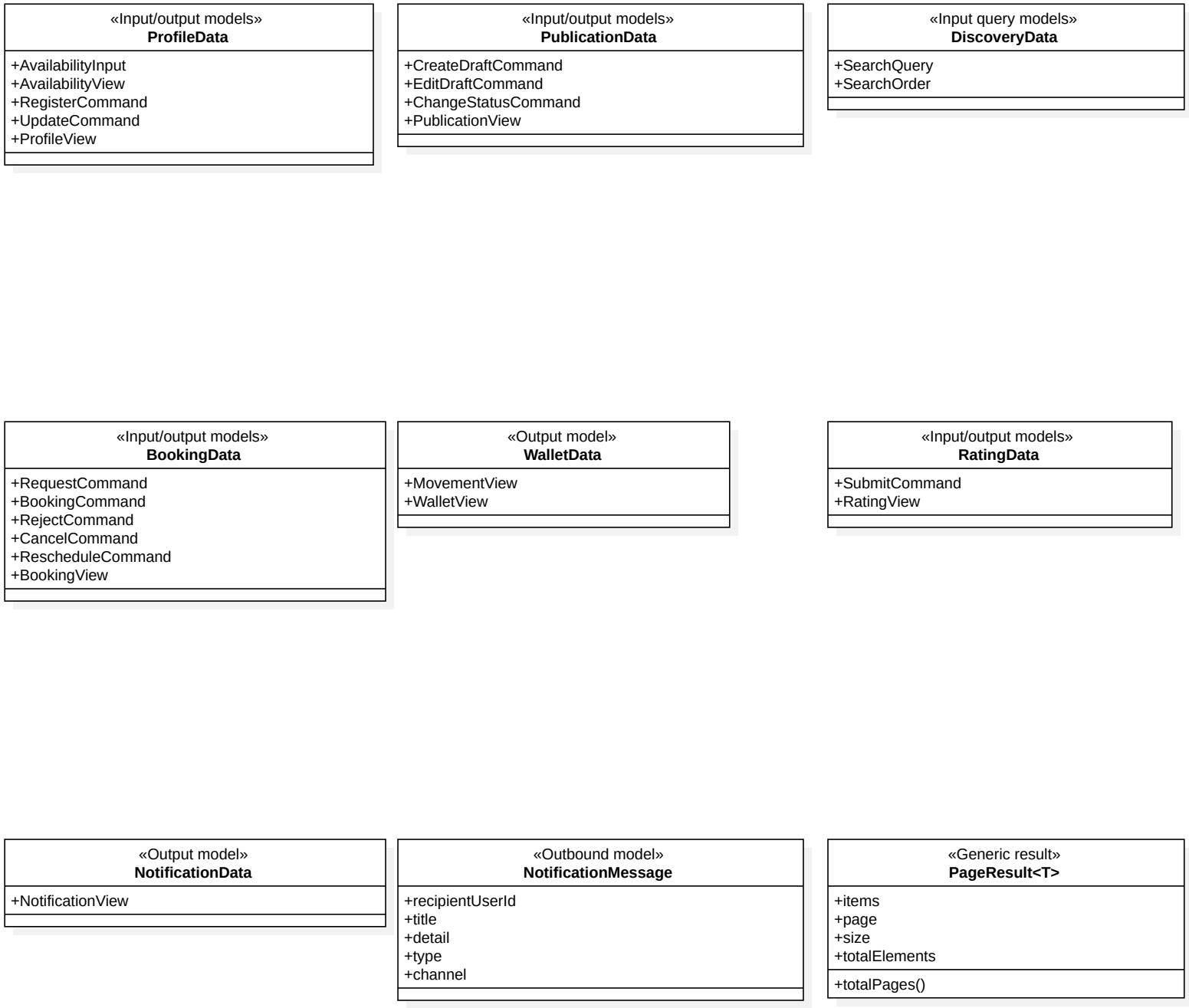
SkillSwap::Application - descubrimiento



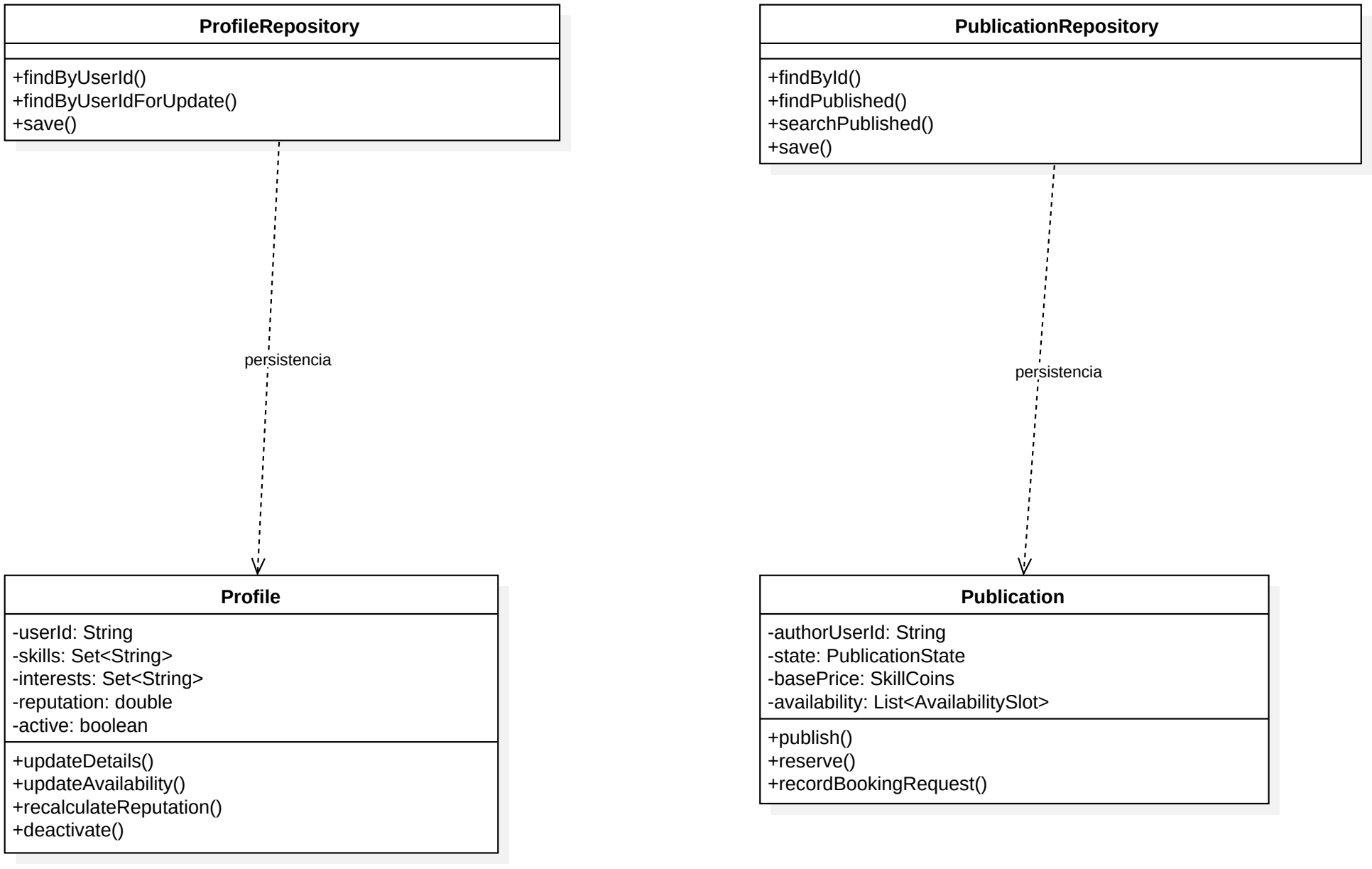

BookingUseCases
+listMyBookings()
+request/accept/reject/pay/cancel()
+propose/acceptReschedule()
+start/complete()
usa modelo
«Input/output models»
BookingData
+RequestCommand
+BookingCommand
+RejectCommand
+CancelCommand
+RescheduleCommand
+BookingView


WalletUseCases
+viewMyWallet()
RatingUseCases
+submit()
NotificationUseCases
+listMine()
+markRead()
+markAllRead()
usa modelo
usa modelo
usa modelo
«Output model»
WalletData
+MovementView
+WalletView
«Input/output models»
RatingData
+SubmitCommand
+RatingView
«Output model»
NotificationData
+NotificationView

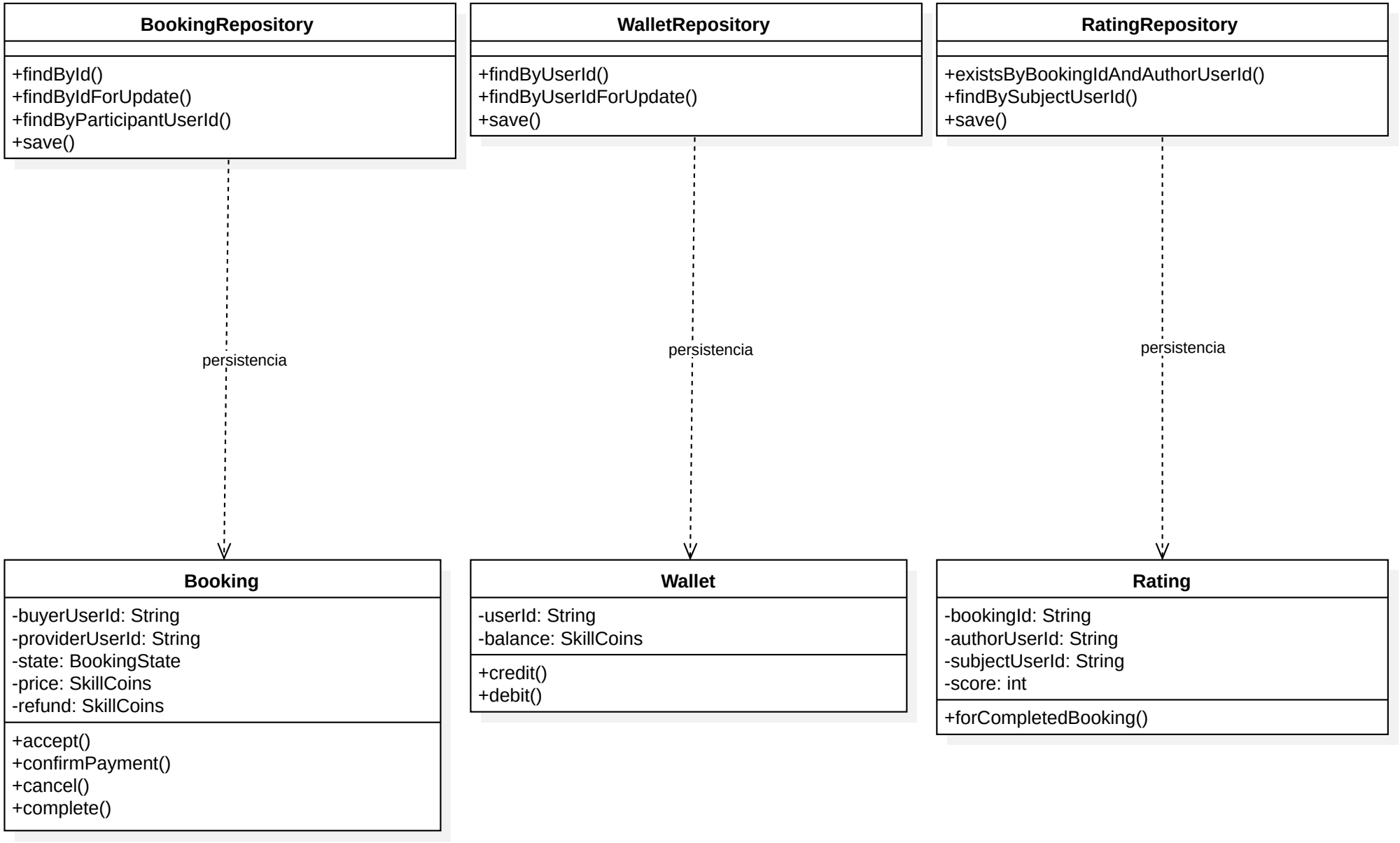
SkillSwap::Application - modelos y resultados compartidos

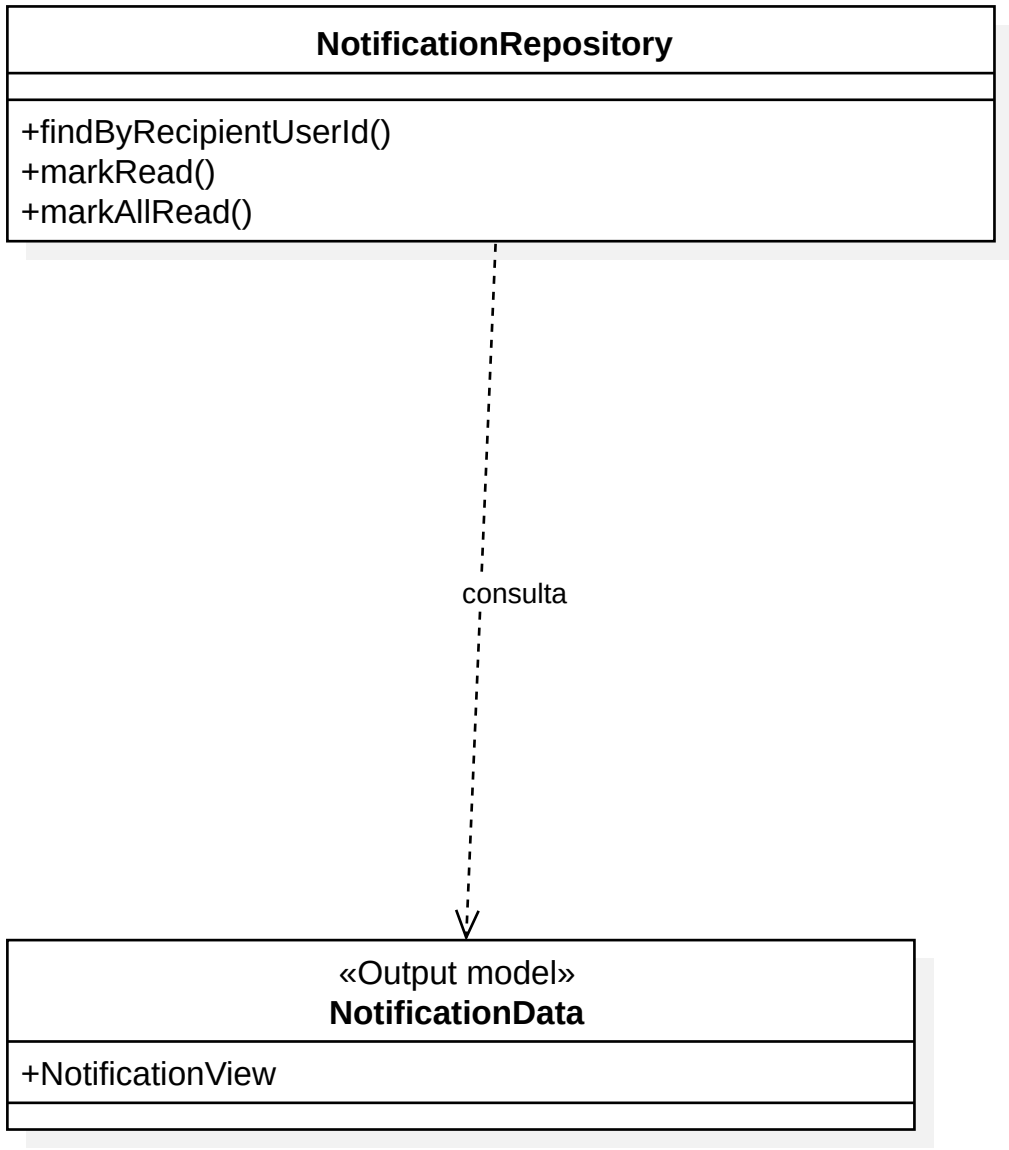


SkillSwap::Application - repositorios de perfil y publicación

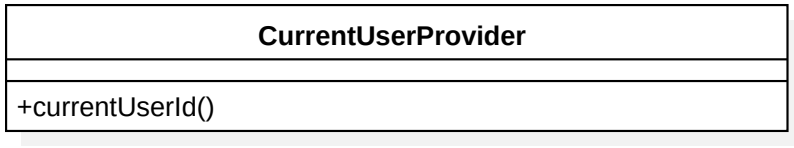


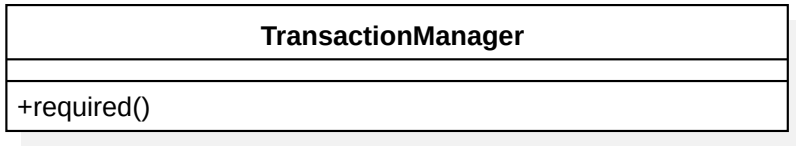
SkillSwap::Application - repositorios de booking, wallet y rating




NotificationRepository
+findByRecipientUserId()
+markRead()
+markAllRead()
consulta
«Output model»
NotificationData
+NotificationView

SkillSwap::Application - usuario autenticado y transacciones





SkillSwap::Application - eventos y notificaciones externas

